

Demo Day · 2026

Task Manager

Gestión colaborativa de tareas en tiempo real

Node.js 24 · Express 5 · React 19 · PostgreSQL 15 · Socket.IO · RabbitMQ

¿Qué es Task Manager?

Una aplicación web full-stack para equipos, con colaboración en tiempo real

El problema que resuelve

- Equipos sin visibilidad compartida del trabajo
- Sin notificaciones cuando algo cambia
- Sin historial de quién hizo qué
- Sin métricas de rendimiento del equipo

La solución

- Tablero Kanban colaborativo en tiempo real
- Emails automáticos de notificación (asíncronos)
- Auditoría completa por tarea
- Dashboard de analíticas + panel de admin

114

Tests pasando

8

Migraciones DB

5

Servicios Docker

35+

Endpoints REST

2

Idiomas (ES/EN)

Stack Tecnológico

Tecnologías modernas de nivel producción

BACKEND

Node.js 24 Express 5 TypeScript strict Knex 3 PostgreSQL 15
Socket.IO 4 RabbitMQ 3 Nodemailer JWT httpOnly bcrypt PDFKit
Yup validation Swagger OpenAPI 3

FRONTEND

React 19 Vite 7 TypeScript BlueprintJS v6 TanStack Query v5 dnd-kit
Recharts react-markdown react-i18next Axios

INFRA / TESTING

Docker Compose Nginx SonarQube node:test Vitest Testing Library

Arquitectura del Sistema

5 procesos separados trabajando en conjunto

FLUJO DE UN REQUEST

- 1 React SPA (Vite / Nginx)**
UI en el navegador — TanStack Query gestiona el caché
- 2 Express 5 API (:3000)**
Rate Limit → CORS → JWT middleware → Yup validation → Controller → Service → DAO
- 3 PostgreSQL 15**
Base de datos relacional. Consultas vía Knex, esquema con migraciones versionadas
- 4 RabbitMQ → Worker**
Publicación asíncrona en cola. Worker consume y envía emails sin bloquear el API
- 5 Socket.IO WebSocket**
Broadcast en tiempo real a todos los clientes conectados

PRINCIPIOS DE DISEÑO

Patrón Result<T>

El backend usa `Result.ok()` / `Result.fail()` en lugar de lanzar excepciones — flujo predecible y fácilmente testeable

Proceso Worker independiente

Los emails nunca ralentizan el API. El worker corre en su propio contenedor Docker consumiendo la cola

TypeScript strict en ambos lados

`exactOptionalPropertyTypes: true` — sin `any` permitido por ESLint

Modelo de Datos

11 tablas · todas las relaciones cubiertas · integridad referencial completa

Tabla	Relación	ON DELETE
users	Raíz (sin FK)	—
categories	Referencia global	—
projects	N:1 → users	CASCADE
project_settings	1:1 → projects (PK=FK)	CASCADE
project_members	N:M users↔projects	CASCADE
tags	N:1 → projects	CASCADE
tasks	N:1 users/projects/categories	CASCADE / SET NULL
task_tags	N:M tasks↔tags	CASCADE
task_assignees	N:M tasks↔users	CASCADE
comments	N:1 → tasks, users	CASCADE
audit_logs	N:1 → tasks, users	CASCADE

TIPOS DE RELACIONES PRESENTES

1:1 — project_settings → projects

El PK de settings ES el FK al proyecto. No puede existir sin él.

1:N — users → tasks, projects → tags...

Un usuario tiene muchas tareas; un proyecto tiene muchos tags.

N:M — project_members, task_tags, task_assignees

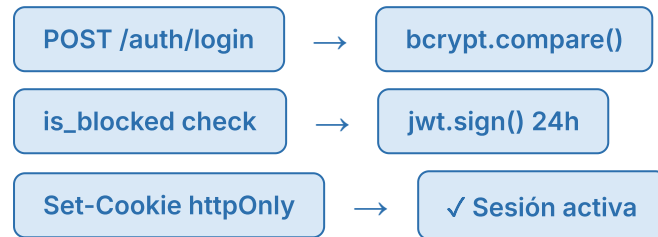
Tablas pivot con PK compuesta y sus propios atributos (role, joinedAt).

categoryId usa **SET NULL** al borrar categoría — **preserva la tarea sin categoría en lugar de eliminarla en cascada**

Autenticación y Seguridad

JWT httpOnly · RBAC · Rate Limiting · Bloqueo en tiempo real

FLUJO DE LOGIN



EN CADA REQUEST PROTEGIDO

- 1 Lee cookie httpOnly**
No accesible desde JavaScript del cliente
- 2 jwt.verify()**
Valida firma y expiración (24h)
- 3 Consulta is_blocked en DB**
Permite expulsión inmediata sin esperar la expiración del token

MEDIDAS DE SEGURIDAD

Rate Limiting

Login: 10 intentos / 15 min
Registro: 5 intentos / hora

RBAC (Control de Acceso por Roles)

Middleware requireAdmin protege todas las rutas /admin. Un USER nunca puede acceder aunque tenga token válido.

Auto-protecciones del Admin

Un admin no puede bloquearse a sí mismo, cambiar su propio rol ni eliminarse.
Evita auto-lockout accidental.

Express Hardening

x-powered-by deshabilitado · CORS restringido a origin específico · Cookies con httpOnly

Funcionalidades

De la tarea individual a la colaboración de equipo

Tablero Kanban

- 3 columnas: Pendiente · En Progreso · Completada
- Drag-and-drop (dnd-kit)
- Prioridad: LOW / MEDIUM / HIGH / URGENT
- Fecha de vencimiento
- Asignación múltiple de usuarios
- Categorías y Tags con color
- Bulk delete · PDF export

Proyectos

- Públicos o privados
- Roles: OWNER / MEMBER
- Unirse / salir / invitar
- Email al unirse o ser añadido
- Gestión de miembros por el owner
- Filtro A-Z / más recientes

Comentarios

- Hilo por tarea (tipo chat)
- Markdown CommonMark
- Avatar Gravatar / iniciales
- Auto-scroll
- Badge de no leídos
- Solo miembros del proyecto

Tiempo Real

- Socket.IO en toda la app
- Cambios de tarea · proyectos · comentarios · admin
- Sin recargar la página

Dashboard

- KPI cards de estado
- Donut chart por status
- Bar chart de carga
- Line chart de tendencia
- Export a PDF

Panel Admin

- Lista de usuarios con stats
- Promover · Bloquear · Eliminar
- Resource Management
- Lead Time por categoría
- Export PDF admin

Panel Admin: Métricas Explicadas

Cuatro vistas analíticas con datos reales de la base de datos

KPI Cards (fila superior)

Calculadas en el backend cruzando users y tasks en memoria:

- **Total usuarios** — COUNT de la tabla users
- **Total tareas** — COUNT de tasks (todas, no solo del admin)
- **Tasa de completado** — $(\text{COMPLETED} / \text{total}) \times 100$, promedio global
- **Tareas urgentes** — COUNT donde priority = 'URGENT'

Donut Chart — Distribución por estado

Suma de PENDING + IN_PROGRESS + COMPLETED de *todos* los usuarios. Usa los mismos colores del tablero. Si no hay tareas en algún estado, ese segmento se omite.

Resource Management (carga de trabajo)

Para cada usuario: cuenta sus tareas **activas** (PENDING + IN_PROGRESS). Muestra barra de progreso relativa al máximo del equipo. Si ≥ 5 tareas activas → badge rojo "Overloaded". Útil para redistribuir trabajo.

Lead Time por Categoría

Mide el tiempo promedio en días desde createdAt hasta que una tarea alcanzó COMPLETED. Agrupado por categoría. Permite identificar qué tipos de trabajo tardan más. Usa audit_logs para saber cuándo se completó cada tarea.

Todas las métricas se actualizan en tiempo real via Socket.IO cuando un admin bloquea, promueve o hay cambios de rol

Gracias

Task Manager — Demo Day 2026

API: localhost:3000

UI: localhost:5173

Docs: /api-docs

RabbitMQ: localhost:15672

Node.js 24 · Express 5 · React 19 · PostgreSQL 15 · Socket.IO · RabbitMQ · Docker